# Salida 1: Relación resuelta

**Aviso de honestidad:** en mi entorno no había Java (`javac` no estaba instalado), así que no pude ejecutar el código. Las salidas están calculadas a mano, paso a paso. Pruébalas en tu compilador online. Todos los programas son un único archivo con `public class Main`.

**Dominio:** como el área futura es NINGUNA, uso dominios cotidianos con temática One Piece. **Lo que se simula:** no hay depurador ni JUnit en un compilador online. Los puntos de ruptura se simulan con trazas, las pruebas unitarias con `comprobar(...)` y los dobles de prueba con valores fijos.

---

# Ejercicio 1: Casos de prueba con frontera (RA3 a, b)

**Conexión:** diseñar casos de prueba (valores normales, frontera e inválidos) y distinguir tipos de prueba. Este ejercicio es una prueba de caja negra.

**Enunciado:** escribe `categoria(int berries)`. Devuelve `"INVALIDO"` si hay menos de 0, `"NOVATO"` de 0 a 99, `"PELIGROSO"` de 100 a 499 y `"SUPERNOVA"` desde 500. En `main`, define un array de entradas y otro de resultados esperados, recórrelos con un `for` e imprime `OK` o `FALLO` por caso.

java

```
public class Main {
    // Devuelve la categoria de un pirata segun su recompensa en berries
    static String categoria(int berries) {
        if (berries < 0) {                // caso limite: valor invalido
            return "INVALIDO";
        } else if (berries < 100) {       // menos de 100
            return "NOVATO";
        } else if (berries < 500) {       // de 100 a 499
            return "PELIGROSO";
        } else {                          // 500 o mas
            return "SUPERNOVA";
        }
    }

    public static void main(String[] args) {
        // Casos de prueba: entradas (frontera incluida) y resultados esperados
        int[] entradas = {-1, 0, 99, 100, 499, 500};
        String[] esperados = {"INVALIDO", "NOVATO", "NOVATO", "PELIGROSO",
"PELIGROSO", "SUPERNOVA"};
```

```
        // Recorremos todos los casos con un bucle for
        for (int i = 0; i < entradas.length; i++) {
            String obtenido = categoria(entradas[i]);              // ejecutamos la
funcion
            String veredicto = obtenido.equals(esperados[i]) ? "OK" : "FALLO"; //
comparamos
            System.out.println("Caso " + (i + 1) + ": entrada=" + entradas[i]
                    + " esperado=" + esperados[i] + " obtenido=" + obtenido + " ->
" + veredicto);
        }
    }
}
```

text

```
Caso 1: entrada=-1 esperado=INVALIDO obtenido=INVALIDO -> OK
Caso 2: entrada=0 esperado=NOVATO obtenido=NOVATO -> OK
Caso 3: entrada=99 esperado=NOVATO obtenido=NOVATO -> OK
Caso 4: entrada=100 esperado=PELIGROSO obtenido=PELIGROSO -> OK
Caso 5: entrada=499 esperado=PELIGROSO obtenido=PELIGROSO -> OK
Caso 6: entrada=500 esperado=SUPERNOVA obtenido=SUPERNOVA -> OK
```

**Explicación:** las dos tablas paralelas (`entradas` y `esperados`) son un caso de prueba: datos de entrada y resultado esperado. Los valores 99/100 y 499/500 son los límites donde suelen esconderse los errores. Los strings se comparan con `.equals`, no con `==`. El `? :` es un `if` abreviado.

---

# Ejercicio 2: Pruebas unitarias automáticas (RA3 f, g)

**Conexión:** pruebas unitarias de funciones y pruebas automáticas con un mini "framework" propio, la idea que luego usa JUnit.

**Enunciado:** escribe `sumarBotin(int[] cofres)`, que suma los cofres. Crea `comprobar(String nombre, int esperado, int obtenido)`, que imprime `[PASA]` o `[FALLA]` y cuenta pruebas y fallos. Lanza 4 pruebas: tres cofres, un cofre, array vacío y una prueba incorrecta a propósito.

java

```
public class Main {
    static int pruebasTotales = 0;   // contador de pruebas ejecutadas
    static int pruebasFallidas = 0;  // contador de pruebas fallidas

    // Funcion a probar: suma el botin de un array de cofres
    static int sumarBotin(int[] cofres) {
        int total = 0;                          // acumulador inicial
```

```
        for (int i = 0; i < cofres.length; i++) {
            total += cofres[i];                 // sumamos cada cofre
        }
        return total;                           // array vacio devuelve 0
    }

    // Mini "framework" de pruebas unitarias automaticas
    static void comprobar(String nombre, int esperado, int obtenido) {
        pruebasTotales++;                       // una prueba mas
        if (esperado == obtenido) {
            System.out.println("[PASA]  " + nombre);
        } else {
            pruebasFallidas++;                  // registramos el fallo
            System.out.println("[FALLA] " + nombre + " esperado=" + esperado + "
obtenido=" + obtenido);
        }
    }

    public static void main(String[] args) {
        comprobar("tres cofres", 600, sumarBotin(new int[]{100, 200, 300}));
        comprobar("un cofre", 50, sumarBotin(new int[]{50}));
        comprobar("sin cofres", 0, sumarBotin(new int[]{}));       // caso limite
        comprobar("prueba erronea a proposito", 10, sumarBotin(new int[]{5, 4}));
// fallara
        System.out.println("Total: " + pruebasTotales + " Fallidas: " +
pruebasFallidas);
    }
}
```

text

```
[PASA]  tres cofres
[PASA]  un cofre
[PASA]  sin cofres
[FALLA] prueba erronea a proposito esperado=10 obtenido=9
Total: 4 Fallidas: 1
```

**Explicación:** una prueba unitaria prueba una sola función de forma aislada y automática, sin que mires la salida a ojo. Los contadores `static` guardan el estado entre llamadas. Un array vacío (`new int[]{}`) es válido y el bucle simplemente no entra. Una prueba que falla no detiene el programa, solo se registra.

---

# Ejercicio 3: Trazas, puntos de ruptura y observación (RA3 c, d, e)

**Conexión:** herramientas de depuración (breakpoint, watch, modificar variables en ejecución). Aquí se simulan con `println` y un `if`.

**Enunciado:** el sombrero de paja tiene `{3, 5, 2, 8, 1}` raciones por día. Acumula el total en un bucle e imprime una traza antes y después de cada suma. Simula un punto de ruptura condicional: si `total > 15`, avisa y corrígelo a 15 (como cambiar una variable en el depurador).

java

```
public class Main {
    public static void main(String[] args) {
        int[] raciones = {3, 5, 2, 8, 1};   // raciones de comida por dia
        int total = 0;                      // variable a vigilar (watch)
        // Bucle con seguimiento paso a paso (simula ejecutar linea a linea)
        for (int dia = 0; dia < raciones.length; dia++) {
            System.out.println("[TRAZA] dia=" + dia + " antes total=" + total);
            total += raciones[dia];         // modificamos el estado
            System.out.println("[TRAZA] dia=" + dia + " despues total=" + total);
            // Punto de ruptura condicional simulado: se activa si total supera 15
            if (total > 15) {
                System.out.println("[BREAKPOINT] total=" + total + " supera 15 en
dia " + dia);
                total = 15;                 // modificamos el valor en "tiempo de
ejecucion"
                System.out.println("[WATCH] total corregido a " + total);
            }
        }
        System.out.println("Total final: " + total);
    }
}
```

text

```
[TRAZA] dia=0 antes total=0
[TRAZA] dia=0 despues total=3
[TRAZA] dia=1 antes total=3
[TRAZA] dia=1 despues total=8
[TRAZA] dia=2 antes total=8
[TRAZA] dia=2 despues total=10
[TRAZA] dia=3 antes total=10
[TRAZA] dia=3 despues total=18
```

```
[BREAKPOINT] total=18 supera 15 en dia 3
[WATCH] total corregido a 15
[TRAZA] dia=4 antes total=15
[TRAZA] dia=4 despues total=16
[BREAKPOINT] total=16 supera 15 en dia 4
[WATCH] total corregido a 15
Total final: 15
```

**Explicación:** en un IDE real pondrías un breakpoint en una línea, verías `total` en la ventana de variables y lo editarías. Aquí imprimimos el estado para "ver" lo mismo. El índice `dia` empieza en 0. El breakpoint se activa dos veces porque tras corregir a 15 la última ración lo vuelve a superar.

---

# Ejercicio 4: Documentar incidencias (RA3 h)

**Conexión:** detectar un fallo, registrarlo con formato fijo (qué función, esperado y obtenido) y verificar la corrección.

**Enunciado:** `contarFuertesErroneo` cuenta cuántos valores de un array son `>= minimo`, pero su bucle se para un elemento antes. `contarFuertes` es la versión correcta. `verificar(...)` imprime una incidencia numerada solo si esperado y obtenido difieren. Prueba con `{90, 40, 70, 100}` y mínimo 70, y con un array vacío.

PROMETEO
by thePower

java

```
public class Main {
    // Version con error: el bucle se detiene un elemento antes
    static int contarFuertesErroneo(int[] fuerza, int minimo) {
        int cuenta = 0;
        for (int i = 0; i < fuerza.length - 1; i++) {   // ERROR: deberia ser
fuerza.length
            if (fuerza[i] >= minimo) {
                cuenta++;
            }
        }
        return cuenta;
    }

    // Version corregida
    static int contarFuertes(int[] fuerza, int minimo) {
        int cuenta = 0;
        for (int i = 0; i < fuerza.length; i++) {        // CORRECTO
            if (fuerza[i] >= minimo) {
```

```
                cuenta++;
            }
        }
        return cuenta;
    }

    static int numIncidencia = 0;   // numerador de incidencias

    // Registra una incidencia si esperado y obtenido difieren
    static void verificar(String funcion, int esperado, int obtenido) {
        if (esperado != obtenido) {
            numIncidencia++;
            System.out.println("INC-" + numIncidencia + " | Funcion: " + funcion
                    + " | Esperado: " + esperado + " | Obtenido: " + obtenido);
        }
    }

    public static void main(String[] args) {
        int[] tripulacion = {90, 40, 70, 100};   // fuerza de cada nakama
        verificar("contarFuertesErroneo", 3, contarFuertesErroneo(tripulacion,
70));
        verificar("contarFuertes", 3, contarFuertes(tripulacion, 70));
        verificar("contarFuertesErroneo (vacio)", 0, contarFuertesErroneo(new
int[]{}, 70)); // caso limite
        System.out.println("Incidencias detectadas: " + numIncidencia);
    }
}
```

text

```
INC-1 | Funcion: contarFuertesErroneo | Esperado: 3 | Obtenido: 2
Incidencias detectadas: 1
```

**Explicación:** el error clásico "off-by-one" (`length - 1`) deja fuera el último elemento (el 100). Una buena incidencia dice dónde, qué se esperaba y qué se obtuvo. Las pruebas correctas no imprimen nada. Con el array vacío, `length - 1` vale -1 y el bucle no entra, así que da 0 por casualidad y esa prueba pasa.

---

# Ejercicio 5: Doble de prueba (stub) para aislar (RA3 i)

**Conexión:** aislar una función de lo impredecible (el dado) sustituyéndolo por valores fijos controlados, que es la idea de un stub o mock.

**Enunciado:** el daño de Luffy es `dado * 10` y devuelve -1 si el dado no está entre 1 y 6. `tirarDado()` es la fuente aleatoria real y no se usa en las pruebas. `danoLuffy(int dado)` recibe el dado por parámetro, así que se puede probar con dados falsos `{1, 6, 0, 7}`.

java

```
public class Main {
    // Funcion real: depende de un dado (aleatorio en un programa real)
    static int tirarDado() {
        return (int) (Math.random() * 6) + 1;   // no determinista
    }

    // Funcion a probar: recibe el dado como parametro para poder sustituirlo
    static int danoLuffy(int dado) {
        if (dado < 1 || dado > 6) {     // caso limite: dado invalido
            return -1;
        }
        return dado * 10;               // dano = dado x 10
    }

    public static void main(String[] args) {
        // Doble de prueba (stub): valores fijos en lugar del dado real
        int[] dadosFalsos = {1, 6, 0, 7};
        int[] esperados = {10, 60, -1, -1};
        // Recorremos cada dado falso y verificamos el resultado
        for (int i = 0; i < dadosFalsos.length; i++) {
            int dano = danoLuffy(dadosFalsos[i]);
            String estado = (dano == esperados[i]) ? "OK" : "FALLO";
            System.out.println("Dado falso=" + dadosFalsos[i] + " dano=" + dano + "
-> " + estado);
        }
    }
}
```

text

```
Dado falso=1 dano=10 -> OK
Dado falso=6 dano=60 -> OK
Dado falso=0 dano=-1 -> OK
Dado falso=7 dano=-1 -> OK
```

**Explicación:** si `danoLuffy` llamara a `tirarDado()` dentro, no podrías predecir su resultado. Al pasar el dado como parámetro, el array `dadosFalsos` hace de doble de prueba. `tirarDado()` queda definido pero sin llamar, para que la salida sea determinista. En un proyecto real se usarían interfaces y bibliotecas de mocks, que quedan fuera del nivel básico.

PROMETEO
by thePower